Invention Disclosure Report: CustomDNS

16 October 2026

CustomDNS is a Java DNS resolver that checks a TTL cache, then a persistent search history, then local records, then real DNS, and shows it all on a desktop GUI and web dashboard. This report follows VIT's Invention Disclosure Format (IDF)-B.

1. Title of the invention

CustomDNS: a multi-tier caching DNS resolver with a TTL cache, persistent search-history lookup (MongoDB with CSV fallback) and browser-history seeding, over a lightweight UDP text protocol.

2. Field / area of invention

Computer networks, specifically DNS name resolution, caching and resolver performance monitoring. Related areas: distributed systems (thread-per-request UDP servers), persistent data stores and human-facing network dashboards.

3. Prior patents and publications from literature

Existing resolvers cache by TTL but none of the common ones combine a TTL cache with a persistent, per-domain search history and an in-app dashboard. The table lists well-known prior art from general knowledge; no formal patent database search was run, so patent numbers are deliberately omitted and a search should precede filing.

Prior art	Type	What it does	Gap relative to CustomDNS
RFC 1034 / RFC 1035 (1987)	Standard	Defines DNS and caching by record TTL	Binary wire format; no persistent usage history
RFC 2308 (1998)	Standard	Negative caching of failed lookups	CustomDNS does not yet cache negatives
BIND 9	Software	Full recursive resolver with in-memory TTL cache	Cache is lost on restart; no search counts or dashboard
dnsmasq	Software	Lightweight forwarder and cache, local hosts file	Fixed-size memory cache; no history store
Unbound	Software	Validating recursive resolver, prefetching	Optimises for speed only; no user-facing history
systemd-resolved / OS stub resolvers	Software	Per-host cache of recent lookups	Opaque to the user; no persistence or analytics
Browser DNS caches (Chrome net-	Software	Short-lived in-browser cache	Not shared, not persistent, no seeding from history

Prior art	Type	What it does	Gap relative to CustomDNS
internals)			
Passive DNS databases	Concept	Store historical DNS answers for security analysis	Collected upstream for forensics, not used to answer queries

4. Summary and background of the invention

CustomDNS answers a DNS query from the cheapest source that has the answer, and remembers every lookup so the next one is cheaper still. A query passes through four tiers in order: an in-memory TTL cache, a persistent search history in MongoDB, a local records file, and finally a real upstream lookup. Each successful lookup is written back to the cache and the history.

Gap. Standard resolvers cache only in volatile memory, so every restart is a cold start, and they keep no per-domain usage record the user can see. Their configuration is also hard to inspect without log parsing.

Novelty.

- A persistent search-history tier that sits between the cache and the records file and repopulates the cache after a restart, with an atomic per-domain search counter and first/last-searched timestamps.
- Seeding that history from the user's own Chrome or Edge browsing history (read from the browser's SQLite file), so the resolver starts warm on the domains the user actually visits.
- Automatic fallback to local CSV files when MongoDB is unreachable, so the resolver keeps working offline.
- A simple pipe-delimited UDP protocol that also reports where each answer came from (CACHE, MONGODB, DNS_LOOKUP) and the remaining TTL, shown live in a Swing GUI and a web dashboard.

5. Objectives of the invention

- Cut lookup latency for repeated domains by serving them from memory.
- Keep learned answers across restarts through a persistent history store.
- Give the user visibility into what was resolved, how often, when and from which tier.
- Remain functional when the database or internet is unavailable.
- Handle many clients concurrently with a simple, inspectable protocol.

6. Working principle (in brief)

The server receives a UDP packet of the form `DNS|REQUEST|<domain>` and hands it to a new thread. The thread tries four sources in order and stops at the first hit:

1. **TTL cache** (RAM). Hit if the entry is younger than its TTL, 60 s by default; expired entries are evicted.
2. **MongoDB search history.** Hit increments the search counter and refills the cache.
3. **Local records file** (`dns_records.txt`, 34 lines). Hit is cached and saved to history.
4. **Real DNS** via the operating system. Hit is cached and saved; failure returns `DOMAIN_NOT_FOUND`.

The reply carries the IP, the source tier and either the TTL left or the search count.

7. Description of the invention in detail

The system is about 4,750 lines of Java 17 and JavaScript, built with Maven. It has a server, two clients, a persistence layer and two user interfaces.

Component	File	Role
DNS server	`DNSServer.java`	Binds UDP port 5454, loads records, starts one `DNSHandler` thread per packet
Request handler	`DNSHandler.java`	Parses the request and runs the four-tier lookup
TTL cache	`DNSCache.java`	Synchronized map of domain to IP, per-entry TTL, hit/miss/eviction counters
History service	`MongoHistoryService.java`	Atomic upsert and counter in MongoDB; mirrors to `dns_search_history.csv` and `dns_queries.csv`
Browser importer	`ChromeHistoryImporter.java`	Copies the Chrome/Edge `History` SQLite file, aggregates visits, resolves and stores each domain
CLI client	`DNSClient.java`	Interactive prompt, 5 s timeout, error handling
Desktop GUI	`DNSGui.java`	Swing app with Server, Client and Query Log tabs
Web dashboard	`DNSWebServer.java`, `index.html`, `app.js`	HTTP on port 8090 with `/api/status`, `/api/history`, `/api/query`
Benchmark	`CacheStressTest.java`	Cold/hot, mixed-traffic and TTL-decay tests

Protocol. Plain text, pipe-delimited: request `DNS|REQUEST|<domain>`; reply `DNS|RESPONSE|<domain>|<ip>|<source>|<n>` where source is CACHE, MONGODB or DNS_LOOKUP and n is TTL seconds left or the search count; error `DNS|ERROR|<domain>|<reason>`.

Cache. Each entry stores the IP, the time cached and its TTL. A read checks expiry, evicts a stale entry and counts it as a miss, so the caller re-resolves. All public methods are synchronized, and hit rate is computed as hits over hits plus misses.

Persistence and fallback. Configuration comes from `MONGODB_URI`, `MONGODB_DATABASE` and `MONGODB_COLLECTION` (defaults: local MongoDB, `custom_dns`, `search_history`). If the database is offline the service logs it and continues on the CSV files, so no query fails because of storage.

Concurrency. The thread-per-request model lets several clients be served at once; responses are sent under a lock on the shared socket.

8. Experimental validation results

On 16 Oct 2026 I compiled the project and ran the bundled `CacheStressTest` against a live `DNSServer` on loopback. The test environment had no MongoDB and no internet, so the server used its CSV fallback and could not reach upstream DNS. Timings are therefore from one short run, not a controlled benchmark.

Test	Result
Mixed workload, 20 queries	12 hits, 8 misses, **60.0% hit rate**
Mean latency, cache hit	**1.50 ms**
Mean latency, cache miss	5.13 ms (hits 3.4x faster)
Cold query `google.com` (local record)	3 ms; second query reported `CACHE HIT` with 60 s TTL
TTL countdown	Remaining TTL fell from 60 s to 59 s across the stream, as designed
Unknown domain	Returned `DOMAIN_NOT_FOUND` in 1–3 ms; no crash
Real upstream lookup (`cloudflare.com`)	Not validated: no internet in the test environment

The single-pair test showed 14 ms for the hit against 3 ms for the cold query (a 0.2x "speedup"), which is first-packet and scheduling noise on one sample; the 20-query stream is the more reliable figure.

Usage data in the project. `dns_search_history.csv` holds 67 domains (65 imported from Chrome history, 2 resolved by the server), and `dns_queries.csv` logs 10 client queries from 6 Oct 2026, showing the history and browser-seeding paths worked on the author's machine.

Not yet measured: behaviour under concurrent load, memory growth, MongoDB-backed latency, and comparison against BIND, dnsmasq or Unbound.

9. Aspects of the invention needing protection

1. The ordered resolution pipeline: TTL cache, then persistent search history, then local records, then real DNS, with write-back to cache and history at each step.
2. Use of a persistent per-domain history (search count, first and last searched time) as a resolver tier that re-warms the cache after restart.

3. Seeding that history from the user's local browser history so the resolver starts warm on frequently visited domains.
4. Dual persistence: database primary with automatic CSV fallback, so resolution never depends on storage availability.
5. A response format that returns the answer source and TTL or search count with every reply, driving live GUI and web dashboards.

The individual parts (TTL caching, UDP, MongoDB) are standard; the claim rests on the combination, so a patentability search is advisable before filing.

10. Technology readiness level

TRL 4: technology validated in a lab. A working prototype with server, clients, persistence and two interfaces exists and was exercised on loopback and on the author's machine, but it has not run in a relevant environment under realistic load. Reaching TRL 5 needs concurrency and soak tests, MongoDB-backed measurements, negative caching and a comparison with an established resolver.

Research			Development			Deployment		
TRL 1	TRL 2	TRL 3	**TRL 4**	TRL 5	TRL 6	TRL 7	TRL 8	TRL 9
			[x]					

11. GitHub Repositories

https://github.com/vrohit1k9/DNS-Client-Server